

Conv2d#

<https://pytorch.org/docs/stable/generated/torch.nn.quantized.Conv2d.html>

Description:

Applies a 2D convolution over a quantized input signal composed of several quantized input planes. For details on input arguments, parameters, and implementation see Conv2d. Only zeros is supported for the padding_mode argument.

Function Signature

```
class torch.nn.quantized.Conv2d(in_channels,  
out_channels, kernel_size, stride=1, padding=0, dilation=1,  
groups=1, bias=True, padding_mode='zeros', device=None,  
dtype=None)[source]#
```

Variables

```
classmethod from_float(mod,  
use_precomputed_fake_quant=False)[source]#
```

Parameters

Code Examples

```
>>> # With square kernels and equal stride
>>> m = nn.quantized.Conv2d(16, 33, 3, stride=2)
>>> # non-square kernels and unequal stride and with padding
>>> m = nn.quantized.Conv2d(16, 33, (3, 5), stride=(2, 1),
padding=(4, 2))
>>> # non-square kernels and unequal stride and with padding and
dilation
>>> m = nn.quantized.Conv2d(16, 33, (3, 5), stride=(2, 1),
padding=(4, 2), dilation=(3, 1))
>>> input = torch.randn(20, 16, 50, 100)
>>> # quantize input to quint8
>>> q_input = torch.quantize_per_tensor(input, scale=1.0,
zero_point=0, dtype=torch.quint8)
>>> output = m(q_input)
```

Notes & Warnings

NOTE Only zeros is supported for the padding_mode argument.

NOTE Only torch.quint8 is supported for the input data type.

Detailed Documentation

Documentation

Applies a 2D convolution over a quantized input signal composed of several quantized input planes.

For details on input arguments, parameters, and implementation see `Conv2d`.

Only zeros is supported for the `padding_mode` argument.

Only `torch.quint8` is supported for the input data type.

`weight` (Tensor) – packed tensor derived from the learnable weight parameter.

`scale` (Tensor) – scalar for the output scale

`zero_point` (Tensor) – scalar for the output zero point

See `Conv2d` for other attributes.

Creates a quantized module from a float module or `qparams_dict`.

`mod` (Module) – a float module, either produced by `torch.ao.quantization` utilities or provided by the user